Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: Mr. Federico Bruzzone





UNIVERSITÀ DEGLI STUDI DI MILANO
Computer Science Department
ADAPT-Lab

Academic Year 2024–2025

COMPUTER SCIENCE DEPARTMENT • ADAPT-LAB

Problem Statement While the concept of specialization is a crucial tool for optimizing generic code in any performance-oriented language, its practical application is often gated by compiler maturity and stability concerns. The lack or instability of specialization in programming languages creates a gap between what is theoretically possible for optimization and what is practically available to the vast majority of the community, putting a performance ceiling on stable code. Specifically for Rust, given its performance focus and the significant utility of trait specialization, the central problem is the lack of a robust, idiomatic and performance-preserving mechanism for implementing trait specialization in stable code.

Proposal To address the general limitations of specialization features in current stable ecosystems, this thesis proposes an innovative approach to the matter by moving the monomorphization and specialization logics to an earlier stage in the compilation process. To prove the effectiveness of this approach, we introduce a metaprogramming-based technique to simulate the behavior of compiler-implemented specialization. By utilizing the metaprogramming capabilities of the language, we can analyze the input and generate the necessary boilerplate and type-checking logic at compile time, effectively allowing a user to define specialized implementations in stable code, avoiding runtime overhead entirely and circumventing the restrictions imposed by unstable compiler features. This approach can be applied to most languages with metaprogramming support and even directly into the compiler frontend of modern languages, as the compiler can simply replicate our approach internally.

Contributions This work provides the following key contributions. A Novel Stable Specialization Technique: the design and implementation of a new, purely metaprogramming-based approach for achieving specialization. Implementation of Reusable Tools: the creation of a Rust procedural macro library that provides an alternative to nightly specialization and an analysis tool that evaluates real-world public Rust codebases to identify common patterns where this stable specialization mechanism could yield substantial benefits.

Example Let's consider a trait `MyTrait` with a method `my_method` and we want to have a generic implementation as well as provide a specialized implementation for the type `i32`. In standard Rust this would not be possible due to the lack of stable specialization features.

To apply our tool to this problem, the default implementation does not need any change compared to standard code. The specialized implementation instead, is annotated with the attribute macro `#[when(...)]`, that lets us specify the conditions under which that implementation should be used. In this case, we want to specialize the implementation of `MyTrait` for the type `i32`, so we write `#[when(T = i32)]`. This will create a specialized version of the `trait` and of the `impl`, through a process we call *monomorphic specialization*.

After doing this, we also need to instruct the method calls to use the correct version of the method. This is done by using the `spec!` macro, which takes as first argument the method call and as second argument the type to specialize for. This is required because in the macro expansion phase the Rust compiler doesn't give us enough information to automatically determine the types of the variables involved in the method call.

The complete code is shown in Listing 1, where you can see the macros in action. As you can probably understand, this approach preserves the soundness of the type system, since after the monomorphization process we have two completely separate versions of the trait and its implementation, that the compiler can easily distinguish and type check independently.

```
trait MyTrait {
    fn my_method(&self);
}
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}
#[when(T = i32)]
impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}
fn main() {
    spec! { 10.my_method(); i32 };   // prints "Specialized for i32"
    spec! { 10.5.my_method(); f64 }; // prints "Generic implementation"
}
```

Listing 1. *Example of the tool usage.*

With this simple example, we have shown how to use the tool to define a specialized implementation of a trait method for a specific type. More complex scenarios are also supported, including multiple specializations, constraints on generic parameters, trait bounds and more.